

GradeMark Pro

Student Grade and Marks Management System

VITyarthi – Build Your Own Project

Project Report

1. Introduction

GradeMark Pro is a desktop application developed in Python using Tkinter. It provides student record management, grade calculation, input validation, JSON storage, and basic class performance statistics.

2. Problem Statement

Manual management of student marks can be time-consuming and may lead to calculation and record-management errors. The project provides a centralized application for maintaining student records and calculating academic performance.

3. Objectives

The objectives are to manage student records, implement CRUD operations, calculate percentage and grades, display class statistics, validate inputs, persist data, and demonstrate modular programming.

4. Functional Requirements

The system provides three major user-facing modules: Dashboard, Student Management, and Marks Calculator. Student Management supports add, edit and delete operations. The Calculator computes percentage, grade and grade points. The Dashboard shows total students, average marks, passed and failed counts.

5. Non-Functional Requirements

Usability: tab-based GUI. Reliability: validation and exception handling. Maintainability: separated modules. Resource efficiency: standard-library Python implementation with lightweight JSON storage.

6. System Architecture

User → Tkinter GUI → application modules → data manager → students.json. Supporting modules handle grading, validation, calculation, dashboard statistics, and student CRUD.

7. Design Diagrams

The repository contains System Architecture, Workflow, Use Case, Sequence, Class/Component, and ER/Storage diagrams under docs/diagrams/.

8. Design Decisions and Rationale

Tkinter was selected for a lightweight desktop GUI. JSON was selected for simple persistent storage. Functionality was separated into modules to improve maintainability and testing.

9. Implementation Details

main.py controls the GUI. student_manager.py handles CRUD operations. data_manager.py reads and writes JSON. grade_utils.py contains grading rules. calculator.py handles calculation. dashboard.py calculates statistics. validation.py validates user input. config.py contains application constants.

10. Screenshots / Results

The screenshots directory contains placeholders indicating the required captures for Dashboard, Students, and Calculator. Replace them with screenshots from the actual running application before final portal submission.

11. Testing Approach

Automated unit tests cover percentage calculation, grade boundaries, and pass/fail boundaries. Manual testing should cover CRUD operations, invalid input, duplicate roll numbers, calculator validation, and persistence.

12. Challenges Faced

Key implementation challenges include separating GUI logic into modules, maintaining consistent student data, validating numeric input, and ensuring JSON records persist between application sessions.

13. Learnings and Key Takeaways

The project demonstrates GUI development, modular programming, CRUD operations, data persistence, validation, basic software architecture, and testing.

14. Future Enhancements

Potential enhancements include authentication, subject-wise marks, GPA/CGPA, charts, export facilities, database storage, advanced search/filtering, and expanded automated testing.

15. References

Python documentation, Tkinter/ttk documentation, JSON module documentation, unittest documentation, and the supplied VITyarthi Build Your Own Project Guidelines.